UNIVERSITÄT BERN

Applied Optimization
Exercise 5 - Line Search Methods

September 18, 2025

What to hand in

A .zip compressed file renamed to `Exercise`*n*`-Group.zip` where *n* is the number of the current exercise sheet. It should contain:

- Hand in **only** the files you changed (headers and source). It is up to you to make sure that all files that you have changed are in the zip.
- A `readme.txt` file containing a description on how you solved each exercise (use the same numbers and titles) and the encountered problems. Indicate what fraction of the total workload each project member contributed.
  Failure to do so will result in you losing a point.
- Other files that are required by your `readme.txt` file. For example, if you mention some screenshot images in `readme.txt`, these images need to be submitted too.
- For the theory exercise, put `TheoryExercise.pdf` with your solutions in the same .zip you submit for the code.
- Submit your solutions to ILIAS before the submission deadline. Late submissions will receive 0 points!
- **Important:** Your submission MUST compile without any issue in order to obtain a passing grade.

Exact line search for the convex quadratic function (0.5 pt)

Consider the convex quadratic function

$$f(x) = \frac{1}{2}x^T Q x + q^T x + c,$$

where $x \in \mathbb{R}^n$, and constant parameters Q is a symmetric positive definite matrix $\in \mathbb{R}^{n \times n}$, q is a vector in $\mathbb{R}^n$, and $c \in \mathbb{R}$.

Given a search direction Δx, compute the exact line search parameter t for an arbitrary point x in the domain of f.

$$t := \arg\min_{s \geq 0} f(x + s\Delta x)$$

Gradient descent with exact line search (0.5 pt)

Consider the unconstrained minimization problem:

$$\text{minimize } \frac{1}{4}x_1^2 + x_2^2$$

starting from point $x^{(0)} = (2,1)$, perform one iteration of the gradient descent algorithm with exact line search. Sketch the function, the line and the update. [1]

What are the values of $\|\nabla f(x)\|^2$ before and after the update $x^{(1)} := x^{(0)} + t^{(0)}\Delta x^{(0)}$?

Programming Exercise: Modified Mass Spring System (1 pt)

In Exercise 3, we setup the mass spring system problem with two different element functions:

$$E_{a,b} = \frac{1}{2}k_{a,b}||\mathbf{x_a} - \mathbf{x_b}||^2$$

and

$$\hat{E}_{a,b} = \frac{1}{2}k_{a,b}(||\mathbf{x_a} - \mathbf{x_b}||^2 - \mathbf{l}_{\mathbf{a,b}}^2)^2.$$

Now let's introduce extra force to the system by fixing certain nodes in the spring graph. Let C be the set of nodes equipped with extra force and p_n be the target coordinates of node n. The force drags the nodes in the spring graph to the target coordinates. Hence, the energy function of the modified mass spring system are:

$$E = \sum_{e(a,b)} E_{a,b} + \sum_{n \in C} \frac{1}{2} w_n ||(\mathbf{x_n} - \mathbf{p_n})||^2$$

and

$$\hat{E} = \sum_{e(a,b)} \hat{E}_{a,b} + \sum_{n \in C} \frac{1}{2} w_n ||(\mathbf{x_n} - \mathbf{p_n})||^2,$$

where w_n is a constant that controls the effect of the force. Here we set $w_n = 100000$. You need to implement the gradient descent method with backtracking line search and find the (local) optimum solutions for the following two different scenarios:

1. Attach the four corner nodes in a $m \times n$ spring graph to target coordinates $(0,0)$, $(2m,0)$, $(0,2n)$ and $(2m,2n)$ respectively.
2. Attach the nodes on the side with index $N_{(i,0)}$ to target coordinates (i, 0) and nodes $N_{(i,n)}$ to coordinates $(i,2n)$, where $i \in [0,m]$.

Regarding the code that is missing, you are requested to fill in:

- Fill the `add_constrained_spring_elements(...)` function in `MassSpringSystemT_impl.hh` to add those penalty terms to the objective function.

[1] here the link to our interactive plotting system
https://slides.cgg.unibe.ch/aopt20/plots/plot2d_create.html

- Complete `eval_f(...)`, `eval_gradient(...)` and `eval_hessian(...)` in the `ConstrainedSpringElement.hh` file.
- Update `eval_f(...)`, `eval_gradient(...)` and `eval_hessian(...)` in the `MassSpringProblem2DSparse.hh` file to take the additional term into account when computing the energy.
- Implement the `solve(...)` function in `Algorithms/GradientDescent.hh` and `Algorithms/backtracking_line_search(...)` in `LineSearch.hh`. As stopping criterion for the gradient descent, you should check that the objective function does not increase, that the norm of the gradient is not greater than some ϵ and set a hard limit on the number of iterations.
  Note: how can you handle cases where the line search goes out of the domain of your objective function? For instance if we are at a negative point for a logarithmic function. C++ will generate a `NaN` in such cases so it's up to you to handle those without crashing your algorithm.

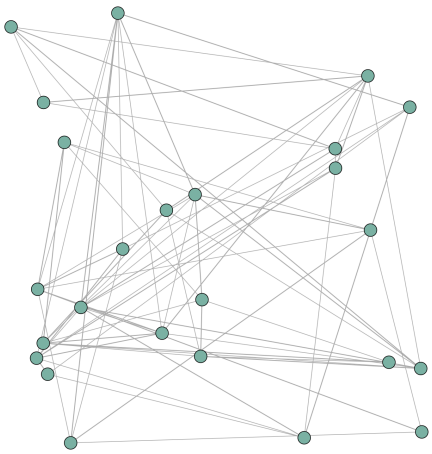

Figure 1: random 4×4 spring graph

For the usage of the executable, please see the `main.cc` in `GradientDescent` folder. You should provide the filename as a parameter so that the program can save the optimized spring graph to the specified files which you can then visualize on this website. Figure 1 is a 4×4 random spring graph.

In the `main.cc`, the gradient descent algorithm runs twice from two different start points. Test the algorithm on mass spring systems of dimension 8×8 of both the spring element without length and with length types. Submit screenshots (8 in total) of the spring graphs (initial and optimized). Record the initial energy value and final energy value at convergence of each test case. Give some comments on how the start points affect the gradient descent method on different function types.

We also provide the unit test for you to check each part of the implementation that needs to be done. Run it by executing `GradientDescent-test`.